SIMATS ENGINEERING

Department of Computer Science and Engineering

Course Code:	CSA15	**Assessment:**	CO5 AT2
Course Name:	Cloud Computing and Big Data Analytics	**Type:**	Operations Optimization
CO Statement:	Apply big data storage, integration, Hadoop, HDFS, and MapReduce concepts for distributed data processing		

Part A: Production Hadoop Operations – Lesson Notes

Monitoring & Observability

A production Hadoop cluster generates thousands of metrics. Monitoring means: (1) collecting these metrics, (2) storing them, (3) displaying dashboards, (4) setting thresholds to trigger alerts.

Component	Key Metrics	Normal Range	Alert Threshold
NameNode	Heap usage, Live/Dead DataNodes, Block count, Edit log size	< 70% heap	> 85% heap
DataNode	Disk usage, Block count, Network I/O, JVM heap	< 80% disk	> 92% disk
MapReduce	Job completion rate, Task failure %, Shuffle sort time	< 5% failure	> 20% failure
HDFS	Under-replicated blocks, Missing blocks, Block replica imbalance	0 under-replicated	> 10 blocks

Lesson Note: Why Monitoring Matters

Without monitoring, you only discover problems when users complain. NameNode heap at 95% → crash → cluster down → data unavailable. Monitoring alerts at 85% gives you time to act (add DataNodes, scale up, optimize)

Backup & Disaster Recovery

NameNode stores the file system namespace (file names, hierarchy, replica locations) in memory and on disk (fsimage + edit logs). If NameNode crashes and no backup exists, you lose the entire namespace — irrecoverable.

Key Concepts:

1. RTO (Recovery Time Objective): How long can the cluster be down? 1 hour? 1 minute?
2. RPO (Recovery Point Objective): How much data loss is acceptable? 1 hour? 1 day?

Backup Strategy	RTO	RPO	Cost	Best For
No backup (risky)	∞ (unrecoverable)	∞	$0	Dev/test only
Daily full backup to external disk	1–2 hours	24 hours	$100/month	Dev, non-critical
Hourly incremental + offsite replication	15 minutes	1 hour	$500/month	Production

HA (active/standby NameNode + ZK quorum)	< 1 minute	0 (near-instant)	$1000+/month	Critical production

High Availability (HA) Design

HA eliminates the single NameNode as a single point of failure. Setup: Active NameNode + Standby NameNode + ZooKeeper quorum (3 nodes). When Active fails, Standby takes over automatically in < 1 minute. Requires shared storage (NFS/QJM) for edit logs.

Cost Optimization

Monthly cluster cost = Hardware + Power + Cooling + Ops

Cost Component	Typical $	Control Method
Hardware (50 nodes × $500)	$25,000 one-time	Buy less, or decommission slow nodes
Annual power & cooling (50 nodes × $1000)	$50,000/year	Efficient hardware, underutilization
Operations & support	$10,000/month	Automation, monitoring, runbooks
Compression (Gzip, Snappy)	$−5,000/month savings	Reduced storage, higher CPU
Tiering (move old logs to cold storage)	$−3,000/month savings	Archive logs > 30 days to S3

Scaling Strategies

Horizontal scaling: Add more DataNodes (linear cost, complexity)

Vertical scaling: Upgrade existing nodes (cap at 10–20TB/node practical limit)

NameNode Capacity Constraint

NameNode heap ∝ number of files. Rule of thumb: 1GB heap ≈ 1M files. If 100M files needed, you need 100GB NameNode heap — impractical. Solutions: (1) Tiering (archive old data), (2) Federation (multiple NameNodes), (3) Consolidate small files.

Part B: Main Scenario – Optimize a 50-Node Production Cluster

Current State Assessment

A company runs a 50-node HDFS cluster processing 10TB/day. Current metrics:

Metric	Current Value	Status
NameNode heap usage	85%	RISKY – near crash threshold
DataNode disk utilization	92%	RISKY – no space for replication
Monthly cost	$50,000	High, no optimization
Last backup	2 weeks ago	UNACCEPTABLE RPO
DataNode failure recovery time	8 hours	UNACCEPTABLE RTO
Monitoring setup	Manual spreadsheets	No real-time alerts
HA setup	None (single NameNode)	Single point of failure

Problem Summary

The cluster is on the edge of failure. NameNode heap crash is imminent. No automated failover. Slow recovery from node failures. Costs unchecked. Backup is stale.

Step 1: Design Monitoring Dashboard

Create a monitoring strategy: what to track, alert thresholds, and dashboard design.

1a. NameNode Metrics

Metrics to track:

- Heap usage (%)
- Live/Dead DataNodes (count)
- Block count (total)
- Edit log size (MB)

Metric	Green	Yellow	Red
NameNode heap %	< 60%	60–80%	> 80%
Dead DataNodes	0	1–2	> 2
Under-replicated blocks	0	< 100	> 100

Fill in your thresholds and actions:

MY MONITORING THRESHOLD DESIGN (Student Response)

Metric	Green (<OK)	Yellow (Warning)	Red (Critical)	Immediate Action
NameNode Heap %	< 60%	60–80%	> 80%	Alert ops; run GC; plan heap increase
Dead DataNodes	0	1–2	> 2	Investigate hardware; check network

Under-replicated Blocks | 0 | 1–100 | > 100 | Trigger HDFS rebalancer immediately
Edit Log Size (MB) | < 512 MB | 512 MB – 1 GB | > 1 GB | Checkpoint NameNode; merge edit logs
DataNode Disk Usage | < 70% | 70–85% | > 85% | Decommission node; add new DataNodes
MapReduce Failure Rate | < 2% | 2–10% | > 10% | Review job logs; check input data

DASHBOARD DESIGN:
I would use Apache Ambari + Grafana. Panels:
- Top panel: NameNode heap gauge (color-coded green/yellow/red)
- Middle row: Live DataNode count, Dead DataNode count, Block counts
- Bottom row: Disk usage per node (bar chart), Job success/failure pie chart
- Alert rules: PagerDuty integration — SMS alert when Red thresholds hit

JUSTIFICATION:
At 85% heap (current state), the NameNode is one large job away from an OOM crash.
Setting the Red threshold at 80% gives the ops team a ~5% safety window to act
before cluster failure. Yellow at 60% means we start planning capacity expansion.

Step 2: Plan Backup & Disaster Recovery

Current state: Last backup 2 weeks ago → RPO of 14 days is unacceptable.

2a. Backup Strategy

Design a backup plan:

Decision	Option A	Option B	Option C
Backup frequency	Daily	Hourly	Real-time (HA)
Storage location	On-cluster NFS	Off-site S3	Shared QJM
RTO	2–4 hours	1 hour	< 1 minute
RPO	24 hours	1 hour	Near-zero
Cost/month	$200	$1000	$2000+ HA infra

Your recommendation: Choose the option that balances cost and business needs.

MY BACKUP & DR RECOMMENDATION (Student Response)

Recommended Strategy: Option B — Hourly Incremental + Off-site S3 Replication

JUSTIFICATION:
Current state: RPO = 14 days (last backup 2 weeks ago) → completely unacceptable for production.

A 14-day RPO means up to 140 TB of data loss (10TB/day × 14 days) — business-ending.

Option A (Daily, $200/month): RPO = 24 hours. Better than current but still risks one full day of 10TB data loss. Acceptable only for non-critical data.

Option B (Hourly incremental, $1000/month) → MY CHOICE:
- RPO: 1 hour → maximum 10GB data loss (10TB/day ÷ 24 hours)
- RTO: 1 hour → restored within one hour of failure
- Cost: $1,000/month is justified for a 50-node production cluster
- Implementation: hadoop distcp to S3 every hour via cron job

Option C (HA, $2000+/month): Ideal but expensive. Recommend as Phase 2 upgrade.

IMPLEMENTATION PLAN:
Step 1 (Week 1): Configure S3 bucket for HDFS snapshots
Step 2 (Week 1): Set up hourly cron: hdfs dfs -distcp hdfs:///data s3a://backup-bucket/
Step 3 (Week 2): Test restore from S3 → verify RTO of 1 hour
Step 4 (Month 2): Budget and plan HA (Active/Standby NameNode + ZooKeeper)

MONITORING OF BACKUP:
- Alert if backup job fails (CloudWatch alarm on S3 PUT failures)
- Alert if backup is > 2 hours old (stale backup detection)

Step 3: Cost Optimization Analysis

Current: 50 nodes × $1000/year ops = $50K/year. Target: Reduce by 20% to $40K/year.

Optimization	Annual Savings	Impact	Your Choice?
Compression (Gzip logs)	$5,000	Reduced storage, higher CPU	Yes / No
Tiering (archive > 30 days to cold)	$3,000	Slower access to old logs	Yes / No
Decommission 5 slow nodes	$5,000	Less capacity, need to rebalance	Yes / No
Auto-scaling (scale down off-peak)	$2,000	Reduced peak hours ops	Yes / No

Calculate total savings from your selections and new monthly cost:

COST OPTIMIZATION SELECTIONS AND CALCULATIONS (Student Response)

SELECTIONS:
✅ Compression (Gzip logs) → Annual savings: $5,000 [SELECTED]
✅ Tiering (archive > 30 days to cold) → Annual savings: $3,000 [SELECTED]
✅ Decommission 5 slow nodes → Annual savings: $5,000 [SELECTED]
❌ Auto-scaling (off-peak) → Annual savings: $2,000 [NOT SELECTED — needs cloud]

CALCULATION:
Current annual cost: $50,000/year ($50,000/year ops × 1)
Total selected savings: $5,000 + $3,000 + $5,000 = $13,000/year
New annual cost: $50,000 − $13,000 = $37,000/year
New monthly cost: $37,000 ÷ 12 = $3,083/month
Savings achieved: 26% reduction (target was 20%) ✅

TRADE-OFF ANALYSIS:
• Gzip compression: CPU overhead increases ~10% but 3:1 storage ratio saves significant disk
• Tiering: Logs > 30 days rarely accessed — acceptable slowdown for cold data
• Decommission 5 slow nodes: Reduce from 50 → 45 nodes; rebalance HDFS with hdfs balancer
Risk: 10% less raw storage → monitor DataNode disk usage post-decommission

IMPLEMENTATION ORDER:
Month 1: Enable Gzip compression on new MapReduce outputs
Month 1: Configure HDFS tiering policy (move >30-day data to S3 Glacier)
Month 2: Decommission 5 underperforming DataNodes; run HDFS rebalancer

Step 4: Address NameNode Heap Pressure

Current heap: 85% at 50M files. At current growth (1M files/month), heap will exceed 90% in ~2 months → crash risk.

Option	Action	NameNode Heap Impact	Timeline
Tiering	Archive logs > 30 days (removes old files)	Reduces heap by ~10%	1 month
Consolidate	Merge small files into larger blocks	Reduces heap by ~15%	Ongoing
Scale	Add second NameNode (Federation)	Splits namespace across 2 NameNodes	2 months setup

Which solution do you recommend? Why?

MY RECOMMENDATION TO ADDRESS NAMENODE HEAP PRESSURE (Student Response)

RECOMMENDATION: Combined approach — Tiering (immediate) + Small File Consolidation (ongoing)

CALCULATION OF URGENCY:
Current: 50M files → 85% heap
Growth: 1M files/month
At 90% heap = NameNode OOM crash risk
Files at 90%: 50M × (90/85) = ~52.9M files → crash in ~2-3 months

IMMEDIATE ACTION (Month 1) — Tiering:
Archive logs older than 30 days to S3 cold storage.
Estimated files removed: ~10M files (logs are the largest namespace consumer)
Heap impact: Reduces from 85% → ~65% immediately
Command: hdfs dfs -mv /data/logs/2024-01-* s3a://cold-bucket/logs/
Cost: $1/TB/month on S3 Glacier vs $50/TB/month on hot HDFS

ONGOING ACTION — Consolidate Small Files:
Problem: 1 small file (< 1MB) uses same NameNode metadata as 1 large file (1GB)
Solution: HAR (Hadoop Archive) or ORC/Parquet file merging in MapReduce
Command: hadoop archive -archiveName logs.har -p /data/logs /data/archive
Effect: Reduce 15M small files → 1M large files → 15× namespace reduction (~15% heap saved)

LONG-TERM (Month 3+) — Federation:
When heap exceeds 75% after cleanup, implement HDFS Federation.
Split namespace: /data/user → NameNode1, /data/logs → NameNode2
Timeline: 2 months to plan, deploy, migrate namespace

Step 5: Implement High Availability (HA)

Current: Single NameNode (SPOF). Proposed: HA with Standby.

Component	Quantity	Purpose
Active NameNode	1	Primary metadata server
Standby NameNode	1	Automatic failover replica
ZooKeeper quorum	3 or 5	Coordinates HA election
Shared edit log storage	NFS or QJM	Both NNs access same metadata

Failover process: Active NameNode fails → ZK detects (30 sec) → Standby promotes (30 sec) → Clients reconnect. Total RTO < 1 minute.

Estimate HA setup cost and timeline:

HA SETUP COST AND TIMELINE ESTIMATE (Student Response)

INFRASTRUCTURE REQUIRED:

Component	Quantity	Unit Cost/month	Monthly Cost
Active NameNode (upgraded)	1	$800	$800
Standby NameNode (new VM)	1	$600	$600
ZooKeeper nodes (3-node)	3	$200 each	$600
QJM (JournalNode cluster)	3	$150 each	$450
Shared Network bandwidth	—	$100	$100
Total additional monthly cost:			$2,550/month

TIMELINE:

Week 1: Provision Standby NameNode and 3 ZooKeeper nodes
Week 2: Deploy JournalNode cluster (QJM) for shared edit log storage
Week 3: Configure HDFS HA in hdfs-site.xml (dfs.nameservices, dfs.ha.namenodes)
Week 4: Test automatic failover: kill Active NameNode → verify Standby promotes in <1 min
Week 5: Run load test with HA enabled; update client configurations (hdfs-site.xml on all nodes)
Week 6: Monitor for 1 week; document runbook; production go-live

FAILOVER VERIFICATION:

Test 1: Manual failover: hdfs haadmin -failover nn1 nn2 → verify Standby becomes Active
Test 2: Simulate crash: kill -9 NameNode process → ZK detects in 30s → Standby promotes in 30s
Total RTO verified: < 1 minute ✅

JUSTIFICATION:

Current SPOF risk: NameNode crash = entire cluster down for 8+ hours (current RTO)
With HA: RTO < 1 minute → 480× improvement in recovery time

Part C: Mini-Problems

Mini-Problem 1: DataNode Failure Scenario

A DataNode holding 12TB of data fails (hardware crash). HDFS detects the failure and begins re-replicating blocks from surviving replicas. Current network capacity: 1 Gbps per DataNode.

Calculate re-replication time and estimate when cluster is back to normal replication factor.

MINI-PROBLEM 1 SOLUTION: DataNode Failure Re-replication Time (Student Response)

GIVEN:
- Failed DataNode data: 12 TB
- HDFS replication factor: 3 (standard)
- Network capacity per DataNode: 1 Gbps = 125 MB/s
- Available DataNodes for re-replication: 49 nodes (50 − 1 failed)

UNDERSTANDING THE SCENARIO:
- When a DataNode fails, HDFS detects it after heartbeat timeout (~10 minutes).
- Blocks that had 3 replicas now have only 2 replicas (under-replicated).
- HDFS must re-replicate these blocks from remaining 2 replicas to reach factor 3 again.
- Only the blocks from the failed node need re-replication (not all 12TB — other 2 copies exist).

CALCULATION:
- Data to re-replicate: 12 TB = 12 × 1024 × 1024 MB = 12,582,912 MB
- Network bandwidth per DataNode: 125 MB/s
- Parallel re-replication: All 49 surviving DataNodes participate simultaneously

- Effective throughput for re-replication:
- With 49 DataNodes each contributing at ~30% of bandwidth (parallel writes):
- Effective throughput = 49 × (125 MB/s × 0.3) = 49 × 37.5 = 1,837.5 MB/s

- Time to re-replicate:
- Time = 12,582,912 MB ÷ 1,837.5 MB/s = 6,849 seconds ≈ 1.9 hours

ANSWER: Cluster returns to full replication factor (3×) in approximately 1.9–2.5 hours.
(Variance due to network contention and block scheduling overhead)

OPERATIONAL NOTE:
- During re-replication, run: hdfs dfsadmin -report | grep "Under replicated"
- Monitor progress in Ambari or Hadoop Web UI (port 9870) → "Under-Replicated Blocks" counter

Mini-Problem 2: NameNode Heap Projection

Current state: 50M files, 60% heap usage. Growth rate: 1.5M files/month. If file count grows linearly, when will NameNode heap exceed 90%? When should you add a second NameNode (Federation)?

MINI-PROBLEM 2 SOLUTION: NameNode Heap Projection (Student Response)

GIVEN:
Current files: 50 million
Current heap usage: 60%
Growth rate: 1.5 million files/month
Rule of thumb: 1 GB heap ≈ 1 million files

CALCULATION — When does heap exceed 90%?

Step 1: Files at 100% heap = 50M / 0.60 = 83.33 million files
Step 2: Files at 90% heap = 83.33M × 0.90 = 75 million files
Step 3: Files to grow: 75M − 50M = 25 million more files
Step 4: Time = 25M ÷ 1.5M/month = 16.7 months ≈ 17 months

ANSWER: NameNode heap will exceed 90% in approximately 17 months from now.

WHEN TO ADD SECOND NAMENODE (Federation)?

Recommendation: Add second NameNode (Federation) when heap reaches 75%.
Files at 75% heap: 83.33M × 0.75 = 62.5 million files
Time to 75%: (62.5M − 50M) ÷ 1.5M/month = 8.3 months ≈ 8 months

ANSWER: Plan and implement HDFS Federation within the next 8 months to avoid heap crisis.

CAPACITY PLANNING TABLE:

Month	Files (M)	Heap %	Action Required
Now	50.0	60%	Start monitoring monthly
Month 4	56.0	67%	Review tiering and small file consolidation
Month 8	62.0	74%	BEGIN Federation planning (design phase)
Month 10	65.0	78%	Start Federation deployment
Month 12	68.0	82%	Complete Federation; heap splits across 2 NNs
Month 17	75.5	90%	Would crash if no action taken

Mini-Problem 3: Compression Trade-off

Implement Gzip compression on 20TB of historical logs. Storage reduction: 3:1 (20TB → 6.67TB). But: CPU overhead for decompression during MapReduce jobs increases job time by 15%.

Is compression worth it economically? Calculate storage cost savings vs. increased compute cost.

MINI-PROBLEM 3 SOLUTION: Compression Trade-off Analysis (Student Response)

GIVEN:
Data size before compression: 20 TB of historical logs
Compression ratio (Gzip): 3:1 → compressed size = 20 ÷ 3 = 6.67 TB
Storage savings: 20 − 6.67 = 13.33 TB saved
Hot storage cost: $50/TB/month
CPU overhead: +15% job time for MapReduce decompression
Current MapReduce compute cost: Assume $0.10/core-hour, 100 cores, 8 hours/day

STORAGE COST SAVINGS:
Savings per month = 13.33 TB × $50/TB = $666.50/month
Annual storage savings = $666.50 × 12 = $7,998/year ≈ $8,000/year

ADDITIONAL COMPUTE COST (15% more CPU time):
Daily compute hours = 100 cores × 8 hours = 800 core-hours/day
Additional hours = 800 × 0.15 = 120 extra core-hours/day
Daily extra cost = 120 × $0.10 = $12/day
Monthly extra cost = $12 × 30 = $360/month
Annual extra cost = $360 × 12 = $4,320/year

NET ANNUAL BENEFIT:
Storage savings: $8,000/year
Additional compute: −$4,320/year
NET BENEFIT: +$3,680/year POSITIVE ✅

ANSWER: YES, compression is economically worthwhile.
It saves $3,680/year net, with break-even in Month 1 (storage savings exceed compute cost immediately).

ADDITIONAL BENEFIT: Compressed data transfers faster over network → MapReduce shuffle phase faster.
RECOMMENDATION: Apply Gzip to cold historical logs. Use Snappy (faster, less compression) for hot data.

Mini-Problem 4: Tiering Strategy

Move logs older than 30 days to cold storage (AWS S3 Glacier, $1/TB/month). Current hot storage cost: $50/TB/month. Hot tier capacity: 20TB. Cold tier capacity: unlimited.

Calculate break-even point: after how many days of cold storage does it become cheaper than hot?

MINI-PROBLEM 4 SOLUTION: Tiering Break-even Calculation (Student Response)

GIVEN:
Hot storage cost: $50/TB/month
Cold storage (S3 Glacier) cost: $1/TB/month
Hot tier capacity: 20 TB
We want to find: After how many days does cold storage become cheaper?

CALCULATION — Break-even Point:

Let d = number of days stored on cold tier.
Cost on hot storage for d days = $50/TB/month × (d/30) months = $50d/30 per TB
Cost on cold storage for d days = $1/TB/month × (d/30) months = $d/30 per TB

This shows cold is ALWAYS cheaper than hot from Day 1.
Ratio: Hot cost is 50× cold cost at any time period.

REAL BREAK-EVEN: Including data retrieval cost from S3 Glacier:
S3 Glacier retrieval: ~$0.01/GB = $10/TB per retrieval
Break-even for retrieval:
Hot monthly savings per TB: $50 − $1 = $49/month
Time to recover retrieval cost: $10 ÷ $49 = 0.2 months = ~6 days

ANSWER: Cold storage (Glacier) becomes cheaper after just 1 day of storage.
Even with retrieval costs factored in, break-even is only 6 days.
For logs older than 30 days (rarely accessed), tiering to Glacier is clearly optimal.

RECOMMENDATION TABLE:

Data Age	Access Frequency	Storage Tier	Monthly Cost/TB
0–7 days	Daily (active)	Hot HDFS	$50
7–30 days	Weekly (queries)	Warm (SSD)	$25
30–90 days	Rarely	S3 Standard	$5
> 90 days	Almost never	S3 Glacier	$1

ANNUAL SAVINGS: Moving 15 TB of logs >30 days to Glacier saves:
(50−1) × 15 = $735/month = $8,820/year

Part D: Problem Bank – 4 Additional Scenarios

Scenario A: Multi-Datacenter Failover

Company wants to replicate the 50-node cluster to a second datacenter 100km away for disaster recovery. Design cross-datacenter replication strategy, network bandwidth requirements, and cost implications. Should data be synchronously or asynchronously replicated? What is RTO/RPO?

SCENARIO A ANSWER: Multi-Datacenter Failover Design (Student Response)

CROSS-DATACENTER REPLICATION STRATEGY:

ARCHITECTURE:
Primary DC (Chennai): 50-node cluster, Active NameNode, all production workloads
Secondary DC (Bangalore, 100km): 50-node cluster, Standby NameNode, disaster recovery

REPLICATION METHOD: ASYNCHRONOUS (Recommended)
Reason: 100km = ~1ms network latency. Synchronous replication would add 1ms to every
HDFS write operation — unacceptable for a 10TB/day ingestion cluster.

Tool: HDFS DistCp (Distributed Copy) scheduled every 15 minutes:
hadoop distcp hdfs://namenode-chennai:8020/data hdfs://namenode-bangalore:8020/data

NETWORK BANDWIDTH REQUIREMENTS:
Daily data ingested: 10 TB/day
Data per 15-min window: 10 TB ÷ 96 = ~104 GB per 15 minutes
Required bandwidth: 104 GB ÷ 900 seconds = ~115 MB/s = ~1 Gbps dedicated link
Recommended: 2 Gbps leased line (100% headroom for spikes)

RTO/RPO WITH THIS DESIGN:
RPO: 15 minutes (last DistCp run → maximum data loss is 15 minutes of ingestion)
RTO: 30–60 minutes (time to promote Bangalore cluster to primary + redirect clients)

COST IMPLICATIONS:
Secondary cluster (50 nodes): $25,000 hardware + $10,000/month ops
2 Gbps leased line: $5,000/month
Total additional cost: ~$15,000/month

SYNCHRONOUS vs ASYNCHRONOUS DECISION:
Synchronous: RPO = 0, but adds latency to every write → NOT recommended for this workload
Asynchronous: RPO = 15 min, no write latency → RECOMMENDED

Scenario B: Federation for Scale-Out

The cluster needs to support 200M files (not possible with single NameNode due to heap limits). Design a Federation setup: how many NameNodes? How to distribute files across namespaces? What is the performance impact? Can you still run cluster-wide MapReduce jobs?

SCENARIO B ANSWER: Federation Design for 200M Files (Student Response)

PROBLEM ANALYSIS:
Single NameNode capacity (practical): ~100M files (100 GB heap)
Required capacity: 200M files
Solution: HDFS Federation with multiple NameNodes

FEDERATION DESIGN:

HOW MANY NAMENODES?
Target: 200M files
Per NameNode safe capacity: 80M files (80% of 100M limit)
NameNodes required: 200M ÷ 80M = 2.5 → Round up to 3 NameNodes

NAMESPACE DISTRIBUTION:
NameNode 1 (NS1): /user/* → User home directories, personal data (est. 80M files)
NameNode 2 (NS2): /data/logs/* → Application logs, events (est. 80M files)
NameNode 3 (NS3): /data/analytics/* → Processed outputs, reports (est. 40M files)

Viewfs configuration in core-site.xml maps each path to correct NameNode.

PERFORMANCE IMPACT:
✅ Benefit: Each NameNode handles 1/3 of namespace → smaller heap per NN → faster metadata ops
✅ Benefit: Parallel namespace operations across 3 NNs → higher metadata throughput
⚠️ Challenge: Cross-namespace operations require additional coordination
⚠️ Challenge: Client-side viewfs configuration must be updated on all nodes

CAN CLUSTER-WIDE MAPREDUCE STILL RUN?
YES — MapReduce reads/writes via viewfs which transparently routes to correct NameNode.
HDFS Federation shares the same DataNodes across all NameNodes (block pool per NS).
Jobs spanning multiple namespaces work correctly — viewfs handles routing automatically.

IMPLEMENTATION TIMELINE:
Month 1: Deploy 2 additional NameNode servers; configure block pools
Month 2: Configure viewfs on all clients and DataNodes
Month 3: Migrate namespace paths; test with non-production jobs
Month 4: Production cutover; monitor heap across all 3 NameNodes

Scenario C: Real-Time Alerting & Auto-Remediation

Design a monitoring + auto-remediation system: when DataNode disk usage exceeds 85%, automatically trigger: (1) alert to ops, (2) pause new block writes to that node, (3) trigger re-replication of critical blocks. What tools would you use? How to test safely?

SCENARIO C ANSWER: Monitoring + Auto-Remediation System (Student Response)

SYSTEM DESIGN: THREE-LAYER APPROACH

LAYER 1 — METRIC COLLECTION:
Tool: Apache Ambari Metrics Collector + Grafana
Metrics polled every 30 seconds:
• DataNode disk usage per node (dfs.datanode.DfsUsed / dfs.datanode.Capacity)
• HDFS blocks (under-replicated, missing, corrupt)
• NameNode heap usage

LAYER 2 — ALERTING:
Tool: Ambari Alerts + PagerDuty webhook
Trigger rule: IF DataNode disk usage > 85% THEN:
Action 1: Send SMS + email alert to ops team (via PagerDuty API)
Action 2: Log event to HDFS audit log: /logs/alerts/disk_alert_$(hostname)_$(date).log
Alert delivery: < 2 minutes from threshold crossing to SMS

LAYER 3 — AUTO-REMEDIATION:
Tool: Custom Python script triggered by Ambari alert webhook

STEP 1: Pause new block writes to overloaded DataNode:
hdfs dfsadmin -setBalancerBandwidth 0 -datanode <affected-node>
(Sets node to "read-only" by halting new block assignments)

STEP 2: Trigger re-replication of critical blocks:
hdfs fsck / -blocks -files -locations > /tmp/fsck_report.txt
hdfs dfsadmin -refreshNodes # Forces NameNode to re-evaluate block placement

STEP 3: Auto-rebalance across healthy nodes:
hdfs balancer -threshold 5 # Moves blocks until no node differs by >5% disk usage

HOW TO TEST SAFELY:
Test 1 (Staging): Fill a test DataNode to 86% → verify alert fires in <2 min
Test 2 (Staging): Verify block writes are paused on overloaded node
Test 3 (Load test): Run with 85% threshold on 1 node; confirm rebalancer moves blocks
Test 4 (Chaos test): Use Netflix Chaos Monkey equivalent to randomly fail nodes; verify recovery
NEVER test auto-remediation on production without a maintenance window.

Scenario D: Cost Benchmarking & Optimization

The company wants to understand their true cost per TB stored. Include: hardware amortization, power, cooling, network, ops salary (allocate 10% of an engineer's time). Compare: (1) on-premise cluster, (2) AWS S3, (3) hybrid (hot on-premise + cold cloud). Which is most cost-effective for 100TB with 2-year horizon?

SCENARIO D ANSWER: Cost Benchmarking — On-Premise vs AWS S3 vs Hybrid (Student Response)

ASSUMPTIONS:
Data: 100 TB | Horizon: 2 years | Access pattern: 30% hot (daily), 70% cold (monthly)
Engineer cost: $120,000/year; 10% time allocation = $12,000/year for Hadoop ops

OPTION 1 — ON-PREMISE CLUSTER (50-node for 100TB at 2TB/node):
Hardware (50 nodes × $2,000): $100,000 one-time (amortized over 2 years = $50,000/year)
Power + Cooling (50 nodes × $1,000/year): $50,000/year
Network: $5,000/year
Ops (10% engineer): $12,000/year
TOTAL 2-YEAR COST: ($50,000 + $50,000 + $5,000 + $12,000) × 2 = $234,000

OPTION 2 — AWS S3 (all 100TB on S3 Standard):
Storage: 100 TB × $23/TB/month × 24 months = $55,200
Data transfer (retrieval, 30TB/month): 30 TB × $0.09/GB × 1024 GB/TB = $2,764/month × 24 = $66,355
No ops salary (managed service): $0
TOTAL 2-YEAR COST: $55,200 + $66,355 = $121,555

OPTION 3 — HYBRID (30TB hot on-premise + 70TB cold on S3 Glacier):
On-premise (15 nodes × $2,000): $30,000 hardware + $15,000/year ops + $5,000/year power
S3 Glacier (70TB × $4/TB/month): $280/month × 24 = $6,720
Retrieval (cold, 5TB/month): $50/month × 24 = $1,200
Engineer (5% time for hybrid): $6,000/year × 2 = $12,000
TOTAL 2-YEAR COST: $30,000 + (15,000+5,000)×2 + $6,720 + $1,200 + $12,000 = $109,920

WINNER: HYBRID OPTION 3 — Most cost-effective at $109,920 over 2 years

COMPARISON SUMMARY:
Option 1 (On-Premise): $234,000 | Best for: High-throughput, data sovereignty required
Option 2 (AWS S3): $121,555 | Best for: Variable workloads, no hardware budget
Option 3 (Hybrid): $109,920 | Best for: Mixed hot/cold workload (RECOMMENDED) ✅

RECOMMENDATION: Implement Hybrid — keep 30TB hot data on-premise for low-latency MapReduce jobs, archive 70TB cold logs to S3 Glacier for compliance and long-term storage.

Evaluation Rubric

Criteria	Wt.	L4 (40–50)	L3 (30–39)	L2 (20–29)	L1 (0–19)
Monitoring & Observability	20%	Comprehensive metrics, realistic alert thresholds, actionable	Good metrics; some thresholds arbitrary	Basic metrics; incomplete alert strategy	Incomplete monitorin design
Cost Optimization	25%	Accurate cost breakdown, realistic savings targets, trade-offs justified	Good analysis; minor gaps in cost modeling	Basic cost analysis; limited optimization ideas	Incorrect or missing analysis
Backup & DR Planning	20%	Credible RTO/RPO targets, feasible failover design, HA understanding	Good DR plan; some RTO/RPO unrealistic	Basic backup strategy; weak failover plan	Incomplete or missin strategy
Scaling & Capacity Planning	20%	Data-driven forecasts, proactive scaling, Federation/HA impact analysis	Good capacity plan; some growth projections weak	Basic capacity analysis; limited forward planning	Missing or unrealistic capacity planning
Documentation & Clarity	15%	Clear dashboards, formulas, step-by-step reasoning, justified decisions	Mostly clear; minor formatting/detail gaps	Understandable but lacks design diagrams/detail	Unclear or poorly organized

Total Marks: 100 (20 + 25 + 20 + 20 + 15)